

UNIT 4. DATABASE DESIGN AND DEVELOPMENT

Subject: Advanced Programming (Programación Avanzada)

Course: 2PVG

Professor: Arnau Rosselló

Year: 2025-2026

Students: Andreu Sánchez Castelló, Guillermo Martorell Hurtado



Index

1.- Database	03
Structure	03
Relations	03
2.- Tasks	04
Maria DB connection	04
SQLite connection	04
ImGui	04
Use our graphic library	04
3.- Manual user	05
How to use our program	05
Main incorporations	06
4.- Conclusions and future work (<i>Post-Mortem</i>)	08



Our database consists of 11 entities:

- Said clear then we use spanish as a main language to all columns and table names instead, but not for the values of them.

To normalize our database, we need to be sure we have all entities related with someone else, an example can be then all tile needs to be on one game and all games need to have a `game_config` value.

[illegible]



2. TASKS

The project was initially designed with the MariaDB DBMS in mind. Later on, we decided to add support for SQLite databases as well.

Since our graphic library is compiled for a 32-bit architecture, this forces our application code to be 32-bit compatible.

Maria DB Connector (Made by Andreu Sánchez)

Modern MariaDB libraries are meant for a 64-bit architecture, but luckily, there are some well-maintained projects of 32-bit connectors that we could use in our project.

This option connects to a remotely-hosted MariaDB instance, that can be managed from the program interface. This connection can be tested using `file connection.cc` (instructions below).

SQLite Connector (Made by Guillermo Martorell)

This connector allows the user to manage a file-based SQLite database that is picked using a file dialog.

Made by using `sqlite3's amalgamation`, that contains, in just a couple of files, all required code to interact with SQLite databases.

ImGui

Our program's visual side was made using ImGui, a command-based graphical interface optimized for performance and widely used in a variety of applications.

ESAT's Graphic Library

The last piece of the puzzle is our school's graphic library, which acts as a graphical backend / renderer for ImGui. The key point here is to prevent the main draw loop from executing SQL queries on each frame, which would stress the database and potentially freeze our program's execution.



3. USER MANUAL

This section will guide you through the required steps to compile, run and use our program. They assume you are on a Windows machine with a usable *Visual Studio Developer Command Prompt (x86)* installed on your system.

Testing the database connection

1. Open a *Developer Command Prompt for VS (x86)* window
2. Go the project's root folder
3. Execute the command: `compilacion.bat connection.cc`

```
-----
ESAT 2026-2027: Programacion Avanzada
-----
Batch script initiated
-----
connection.cc
DatabaseLite.cc
DatabaseMaria.cc
nfd_win.cpp
Generating Code...
-----
Batch script finished
-----
```

4. Execute the file: `connection.exe`

```
Practica_Unit4_BBDD_Rummikub>connection.exe
[1] Initializing MariaDB...
[2] mysql_init() OK
[3] Verification SSL desactivate.
[4] THIS IS WORKING!
[5] MariaDB: 10.11.18-MariaDB-0+deb12u1

[6] Tables from practica-rummi database:
- campeonatos
- config_partidas
- fichas
- partic_partidas_camp
- participantes_campeonatos
- partidas
- partidas_campeonatos
- partidas_usuarios
- posicion_tablero
- posicion_tablero_campeonato
- usuarios

[7] Close in it.
```

If we want to test the connection to the SQLite database (file must be located under `./SQLite/rummi.db`) follow the same steps but with the `connectionLite.cc` file:

```
> compilacion.bat connectionLite.cc
> connectionLite.exe
```

Using the program

Compilation

The script `compilacion.bat` also allows us to compile the main program:

```
> compilacion.bat main.cc
```

If this command exits without errors, the program has been successfully compiled and a file `main.exe` should have been created as the main executable.

Execution and database selection



Launch our program's entry point:

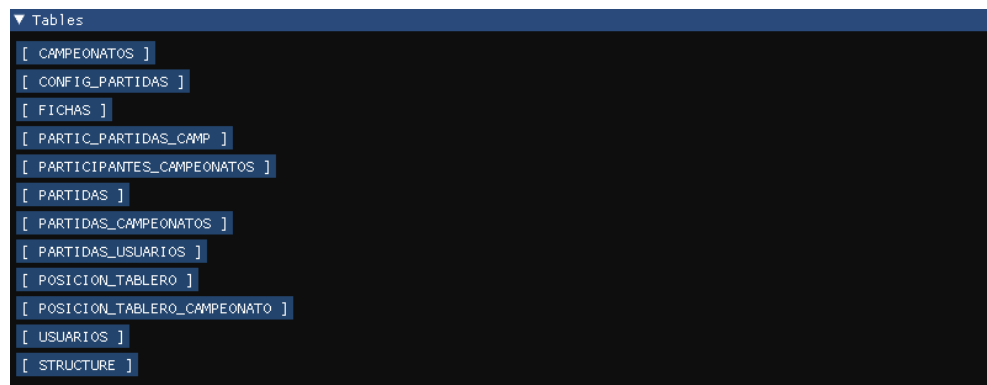
```
> main.exe
```

You will be prompted to choose between the MariaDB connection, the SQLite connection, or exit the program.

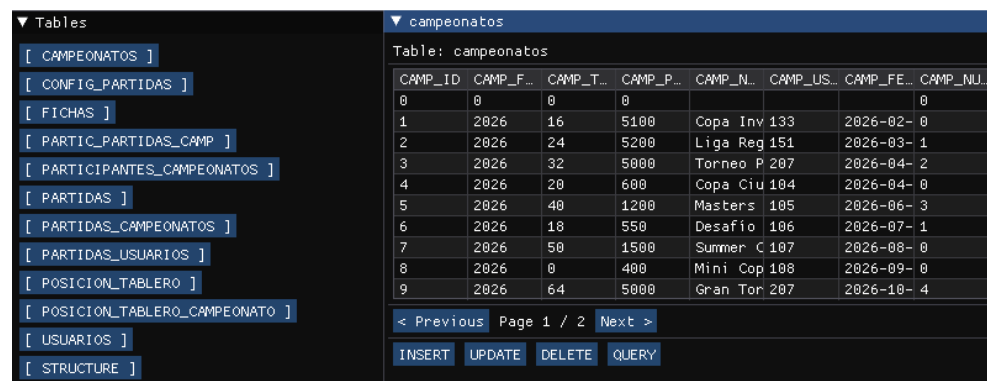
```
Select a database provider:
[1] MariaDB
[2] SQLite
[0] Quit
> 
```

MariaDB

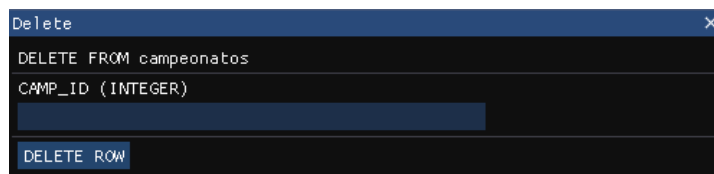
If we pick this option we will go directly to our program's main part, the database visor, listing all the tables contained in the Database and an additional button that shows the database structure.



Clicking on any of them will list all their data. Additionally, we see some buttons to perform various operations on the database.



DELETE allows us to remove rows. The first column is used as the removal key.





INSERT and UPDATE bring you to the insert and edit forms, respectively. The first field is, again, used as a key for the update operation.

Insert

INSERT INTO campeonatos

CAMP_ID (INTEGER)

CAMP_ID

CAMP_FECHA_COMIENZO (INTEGER)

CAMP_FECHA_COMIENZO

CAMP_TOTAL_INSCRIPCIONES (INTEGER)

CAMP_TOTAL_INSCRIPCIONES

CAMP_PUNTOS_GANADOR (INTEGER)

CAMP_PUNTOS_GANADOR

CAMP_NOMBRE_CAMPEONATO (TEXT)

CAMP_NOMBRE_CAMPEONATO

CAMP_USR_CREADOR_CAMP (TEXT)

CAMP_USR_CREADOR_CAMP

CAMP_FECHA_FINALIZACION (TEXT)

CAMP_FECHA_FINALIZACION

CAMP_NUM_USR_BLOQUEADOS (INTEGER)

CAMP_NUM_USR_BLOQUEADOS

INSERT ROW

Update

UPDATE campeonatos

CAMP_ID (INTEGER)

CAMP_ID

CAMP_FECHA_COMIENZO (INTEGER)

CAMP_FECHA_COMIENZO

CAMP_TOTAL_INSCRIPCIONES (INTEGER)

CAMP_TOTAL_INSCRIPCIONES

CAMP_PUNTOS_GANADOR (INTEGER)

CAMP_PUNTOS_GANADOR

CAMP_NOMBRE_CAMPEONATO (TEXT)

CAMP_NOMBRE_CAMPEONATO

CAMP_USR_CREADOR_CAMP (TEXT)

CAMP_USR_CREADOR_CAMP

CAMP_FECHA_FINALIZACION (TEXT)

CAMP_FECHA_FINALIZACION

CAMP_NUM_USR_BLOQUEADOS (INTEGER)

CAMP_NUM_USR_BLOQUEADOS

UPDATE ROW

Last, but not least, QUERY brings you to the “free-style” query menu, that allows the user to run arbitrary queries.

Query to execute:

```
SELECT CAMP_FECHA_COMIENZO AS "FECHA_COMIENZO",
       SUM(CAMP_TOTAL_INSCRIPCIONES) AS "INSCRIPCIONES"
FROM `campeonatos`
GROUP BY "FECHA_COMIENZO"
```

Execute!

Query Result

Query result:

FECHA_COMIENZO	INSCRIPCIONES
NULL	NULL
123	246
333	444
1234	NULL
2026	299
11111	NULL
123123132	123123



CONCLUSIONS AND FUTURE WORK (*Post-Mortem*)

Analyze

After all, I think was a pretty but small PHPMyAdmin what we have here, we can dynamically show all the data and even be sure this structure is working.

Obviously we don't show anything in a static way, which I can say then structure screen was so difficult to visualize, that's why we only show the tables and their columns with types, but nothing more.

On last practices I don't even do anything particular with the global variables, I use to use them a lot, but I decide here just try to correct this, even know then all the booleans works as a flow control statements. I know this can be updated as a major upgrade but it was easier for me doing it this way, even though all local functions can establish all code there.

I need to mention the Database class (including their header, and their implementation). I prefer use this as a class because I want to separate everything on some forms, Database class, and main "class", Database include it for a layer on SQL statements and Main to take this data and modify their response,

After all, I think we should use a cool way to implement everything, but I know some code of Main can be moved to another class or file, to just move unnecessary logic onto the main file too, but I decided not to move forward on it.

It is the same as other implementations like for SQLite, we can encapsulate the code but we don't know the exact way to do it after all.